



NAME : HAMDAN ALI

WEEK 7 : Day 33

DAY 33

TASK : Screenshoots Of Output

CODEBASE Of the Work

```
Week7 > day33 > frontend > {} package.json > ...
1  {
2    "name": "taskflow-frontend",
3    "private": true,
4    "version": "0.1.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "preview": "vite preview"
10   },
11   "dependencies": {
12     "axios": "^1.7.2",
13     "react": "^18.3.1",
14     "react-dom": "^18.3.1"
15   },
16   "devDependencies": {
17     "@vitejs/plugin-react": "^4.3.1",
18     "vite": "^5.4.0"
19   }
20 }
21
```

```
PS D:\internship ( HindukushSoft Technologies )\Week7\taskflow-frontend> npm run dev
> vite

VITE v8.2.2 ready in 2962 ms
  ➔ Local:   http://localhost:5173/
  ➔ Network: use --host to expose
  ➔ press h + enter to show help
```



Full PROJECT View

TaskFlow

Full-stack CRUD — React talking to the Laravel API

New Task

Title

Description

Status

Pending

Priority

Medium

Due date

mm/dd/yyyy

Create Task

My Tasks

Search by title...

All statuses

All priorities

Add email notifications

MEDIUM

Send users an email reminder when a task they own is approaching its due date.

Pending Due 2026-09-30T00:00:00.000000Z

Edit Delete

Clean up unused CSS classes

LOW

Audit the stylesheet and remove any selectors no longer referenced in the components.

Completed

Edit Delete

Design new dashboard layout

HIGH

Create wireframes for the redesigned admin dashboard, focusing on a cleaner sidebar and quicker access to reports.

In Progress Due 2026-09-12T00:00:00.000000Z

Edit Delete

Update API documentation

LOW

Document all new CRUD endpoints, including sample requests and responses, in the project README.

Pending Due 2026-09-18T00:00:00.000000Z

Edit Delete

Design new dashboard layout

LOW

Create wireframes for the redesigned admin dashboard, focusing on a cleaner sidebar and quicker access to reports.

Pending Due 2026-09-27T00:00:00.000000Z

Edit Delete

Update API documentation

LOW

Document all new CRUD endpoints, including sample requests and responses, in the project README.

Completed Due 2026-08-29T00:00:00.000000Z

Edit Delete

Optimize database queries

HIGH

Review slow queries on the tasks table and add appropriate indexes to improve response time.

In Progress Due 2026-08-28T00:00:00.000000Z (overdue)

Edit Delete

Plan sprint retrospective

MEDIUM

Gather feedback from the team on what went well and what could be improved in the next sprint.

Completed

Edit Delete

Migrate legacy user data

MEDIUM

Write a migration script to move existing user records into the new normalized schema.

Pending Due 2026-09-18T00:00:00.000000Z

Edit Delete

Prepare weekly progress report

MEDIUM

Summarize completed tasks, blockers, and next steps for the internship supervisor meeting.

Pending Due 2026-09-01T00:00:00.000000Z (overdue)

Edit Delete



EXPLANATION OF THE TASK

DAY 33

TASKFLOW

Day 33

Full-Stack CRUD — React + Laravel Eloquent

Week 7 — Day 33

Task: Connect React Frontend with Laravel Backend for CRUD

Learning Topics: Full-Stack CRUD

Stack: Laravel 12 (Eloquent), React 18, Axios

1. Introduction And Objective

Day 33's objective was to connect the React frontend to the Laravel backend for full Create, Read, Update and Delete operations — turning the read-only dashboard from Day 32 into a genuinely usable, interactive task manager. This is the task where the project stops being a demo and starts behaving like a real application: users can add tasks, edit them, mark them complete, and remove the ones they no longer need, all backed by a real database through the API.

I treated this as an opportunity to demonstrate a solid understanding of Laravel's Eloquent ORM (as referenced in the official Laravel documentation) on the backend, and of state management patterns for mutations on the frontend — specifically, how to keep the UI in sync with the server after every create, update, or delete.

2. Backend: Completing the RESTful Resource

2.1 Resourceful Routing

The Day 31/33 controller only implemented `index()` and `show()`. I completed it with `store()`, `update()` and `destroy()`, and switched the route definition to Laravel's `Route::apiResource()`, which automatically wires up all five conventional actions to the correct HTTP verb and URI in a single line — this is far less error-prone than hand-registering five separate routes, and it is the pattern the official Eloquent/routing documentation recommends for resource-style controllers.

Verb	URI	Controller Method	Purpose
GET	/api/tasks	index	List tasks
POST	/api/tasks	store	Create a task
GET	/api/tasks/{id}	show	Read one task
PUT	/api/tasks/{id}	update	Edit a task
DELETE	/api/tasks/{id}	destroy	Delete a task

2.2 Form Requests for Validation

Rather than validating inline inside the controller, I created dedicated `StoreTaskRequest` and `UpdateTaskRequest` classes. This keeps the controller focused on orchestration (call Eloquent, return a response) while the Form Request owns the rules, custom error messages, and — importantly — overrides `failedValidation()` so that even validation failures are returned in the exact same `{ success, message, errors }` JSON envelope used everywhere else in the API. Consistency here matters: the frontend can write one single piece of error-handling logic that works for every endpoint.

2.3 Eloquent in Action

Every mutating action uses Eloquent's expressive API rather than raw SQL: `Task::create()` for inserts, `$task->update()` for edits, and `$task->delete()` for removals — each of which Laravel translates into the correct, safely parameter-bound SQL query. Using `$request->validated()` (rather than the raw `$request->all()`) when creating or updating ensures that only fields which passed validation are ever written to the database, which is both a data-integrity and a security safeguard.

3. Frontend: Driving CRUD from React

3.1 Extending the Service Layer

`taskService` gained three new methods — `createTask`, `updateTask`, `deleteTask` — that map directly onto the new backend endpoints. The `useTasks` hook was extended with matching mutation helpers (`createTask`, `editTask`, `removeTask`) that call the service and then reconcile local state with the server's response, so the UI is always an accurate reflection of what actually exists in the database rather than an optimistic guess that could drift out of sync.

3.2 A Single, Reusable Form for Create AND Edit

Instead of building two separate forms, `TaskForm` accepts an optional `initialTask` prop. When it is null the form behaves as a "create" form; when it holds a task object, the same form pre-fills its fields and behaves as an "edit" form. This is a common, battle-tested React pattern that avoids duplicated markup and logic between two nearly identical components.

3.3 Inline Validation Error Display

When the backend responds with a 422 (validation failure), the exact field-level messages Laravel generated are displayed directly beneath the relevant input — not as a generic toast, but

attached to the specific field that failed. This gives the user precise, actionable feedback and is a direct, practical application of this week's "validation" learning topic, previewing the deeper error-handling work completed in Day 35.

3.4 Safe Deletes

Deleting is irreversible, so TaskItem asks for confirmation via `window.confirm()` before calling the delete endpoint, and shows a "Deleting..." state on the button while the request is in flight so the user gets immediate feedback and cannot accidentally double-click and fire two delete requests.

4. Sample Code Walkthrough

Laravel — creating a task from validated input

```
public function store(StoreTaskRequest $request): JsonResponse
{
    $task = Task::create($request->validated());

    return response()->json([
        'success' => true,
        'message' => 'Task created successfully.',
        'data'    => new TaskResource($task),
    ], 201);
}
```

React — submitting the form and handling validation errors

```
try {
    await onSubmit(form);
    if (!initialTask) setForm(EMPTY_TASK);
} catch (err) {
    if (err.response?.status === 422) {
        setErrors(err.response.data.errors || {});
    } else {
        setErrors({ general: [err.response?.data?.message || 'Something went wrong.'] });
    }
}
```



5. How to Run This Locally

1. Backend: copy the updated app/, database/ and routes/ files into your Laravel project, run `php artisan migrate`, then `php artisan serve`.
2. Frontend: `cd frontend` And `npm install` And `npm run dev`.
3. Open the app, add a task using the form at the top, then try editing and deleting it to exercise the full CRUD cycle end-to-end.

6. Challenges And How I Solved Them

- Keeping the create-and-edit UI as a single form without it becoming confusing — solved by treating the presence of `initialTask` as the single source of truth for the form's mode, driven entirely by props.
- Preventing stale data after a mutation — after every create, the hook re-fetches the current page from the server (so pagination totals stay correct); after an update, the specific task in local state is replaced with the server's authoritative response rather than trusted client-side data.

7. Key Takeaways

- `Route::apiResource()` is the idiomatic way to expose a full CRUD resource in Laravel.
- Form Requests keep validation logic out of controllers and make error responses consistent across the whole API.
- A single, prop-driven form component can safely handle both create and edit without duplication.
- Always resolve local UI state from the server's response after a mutation — never assume the client's optimistic view is correct.

WHERE TO FIND THE CODE

All source code for this task is included alongside this document inside the `backend/` and `frontend/` folders. See their `README.md` files for exact setup steps.

THANK YOU!